

PROTOCOLO UDP CONFIABLE PARA RED NEURONAL DISTRIBUIDA

Proyecto de Redes y Comunicaciones

18 de mayo de 2026

Índice

1. Descripción General	2
2. Configuración de la Red Neuronal	2
3. Arquitectura General	2
4. Responsabilidades del Sistema	3
4.1. Python	3
4.2. Servidor Principal	4
4.3. Workers	4
5. Transferencia Confiable de Datos (RDT)	5
6. Características del Protocolo RDT	5
7. Parámetros del Protocolo	6
8. Formato del Datagrama UDP	6
8.1. Estructura del datagrama	6
8.2. Descripción de campos	7
9. Tipos de Datagrama	7
10. Tipos de Objeto del Protocolo	7
11. Capa de Aplicación y Objetos Serializados	8
12. Identificadores de Transferencia	8
13. Campos Usados por Cada Tipo de Datagrama	8
14. Integridad de Datagramas	9
15. Números de Secuencia, ACK y Ventana	9
16. Timeouts y Reintentos	10

17.Simulación de Fallos	10
18.Estados del Emisor y Receptor	11
18.1. Estados del Emisor	11
18.2. Estados Lógicos del Receptor	11
19.Sesiones Master–Worker	11
20.Secuencia de Comunicación	11
21.Distribución del Batch y Objetos	12
22.Serialización de Objetos	13
23.Flujo General del Sistema	13
24.Promedio de Gradientes y Actualización de Pesos	14
25.Integración Python y C++	14
26.Posibles Mejoras Futuras	15
27.Tecnologías Utilizadas	15
28.Conclusión	15

1. Descripción General

El objetivo del proyecto es implementar un sistema de Red Neuronal Distribuida (DNN) donde el cálculo de backpropagation se reparte entre varias computadoras o terminales y el servidor principal combina los resultados mediante el promedio de gradientes.

- La red neuronal se define y entrena en Python.
- La ejecución principal se inicia desde un archivo `.py`.
- Python llama a una biblioteca distribuida en C++ mediante `pybind11`.
- La biblioteca C++ implementa la serialización de objetos, la comunicación UDP y el protocolo RDT.
- La comunicación entre nodos utiliza UDP.
- La confiabilidad se implementa manualmente mediante un protocolo RDT (Reliable Data Transfer) basado en Go-Back-N puro con ACK acumulativo.

La arquitectura del sistema está compuesta por:

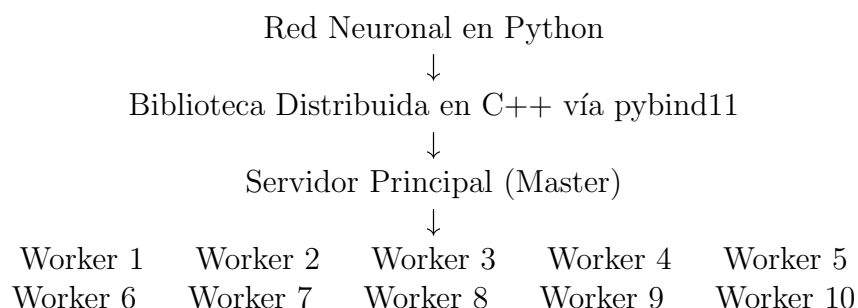
- Un servidor principal (Main Server / Master)
- Diez servidores trabajadores (Workers)
- Un programa o módulo de distribución de datos invocado por el servidor principal

El servidor principal toma un único batch global y lo divide en 11 partes iguales: una parte queda en el master y las otras 10 partes se envían a los workers. El forward pass se ejecuta únicamente en el master. Luego el master construye un objeto serializado `WORK_ASSIGNMENT` para cada worker, que contiene su partición del batch y todos los tensores necesarios para que pueda ejecutar backpropagation correctamente. Finalmente, recibe un objeto `GRADIENT_RESULT` desde cada worker, promedia los gradientes elemento a elemento y actualiza los pesos de la red neuronal principal.

2. Configuración de la Red Neuronal

La configuración de la red neuronal es la siguiente:

3. Arquitectura General



Parámetro	Valor
Capas ocultas	4
Neuronas por capa oculta	200
Matrices base de pesos	4 matrices completas
Tamaño de cada matriz oculta	200x200
Número de batches	1 batch global
División del batch	11 particiones
Épocas	1
Nodos de backpropagation	1 master + 10 workers
Objeto master → worker	WORK_ASSIGNMENT: partición del batch, pesos, activaciones, preactivaciones, etiquetas, derivadas necesarias y metadatos
Objeto worker → master	GRADIENT_RESULT: gradientes calculados por el worker y metadatos de reconstrucción
Protocolo de comunicación	UDP
Capa de confiabilidad	Go-Back-N RDT con ACK acumulativo
Tipo de dato principal	float32
Formato binario de tensores	little-endian
Endianness del protocolo	big-endian / network byte order
Tamaño de ventana Go-Back-N	8 datagramas
Timeout de retransmisión	500 ms
Máximo de reintentos	5

4. Responsabilidades del Sistema

4.1. Python

Python será responsable de:

- Cargar el dataset
- Definir la red neuronal
- Ejecutar la lógica de entrenamiento de la red neuronal
- Seleccionar el único batch global de entrenamiento
- Ejecutar el forward pass únicamente en el master
- Preparar las particiones del batch y los tensores intermedios necesarios para backpropagation
- Construir los objetos `WORK_ASSIGNMENT` que serán enviados a los workers

- Ejecutar backpropagation local en el master sobre la partición 0
- Llamar a la biblioteca distribuida en C++ desde el archivo principal `.py`
- Recibir gradientes promediados
- Actualizar la red neuronal principal usando los gradientes combinados

La integración entre Python y C++ se realizará mediante `pybind11`.

4.2. Servidor Principal

El servidor principal será responsable de:

- Invocar el programa de distribución del batch
- Dividir el batch global en 11 partes iguales
- Mantener la partición 0 para el master
- Enviar las 10 particiones restantes a los workers
- Recibir desde Python los objetos `WORK_ASSIGNMENT` serializables
- Serializar cada objeto de aplicación como una secuencia binaria opaca para la capa RDT
- Construir datagramas UDP
- Administrar ventana Go-Back-N
- Enviar objetos serializados a los workers
- Administrar ACK y retransmisiones
- Recibir objetos `GRADIENT_RESULT` desde los workers
- Reentregar a Python los gradientes contenidos en los objetos reconstruidos
- Promediar los gradientes del master con los gradientes recibidos
- Retornar los gradientes promediados hacia Python para actualizar los pesos

4.3. Workers

Los workers serán responsables de:

- Recibir datagramas UDP
- Verificar integridad
- Reconstruir fragmentos en orden
- Detectar pérdidas y duplicados
- Descartar datagramas fuera de orden

- Recibir y reconstruir un objeto `WORK_ASSIGNMENT`
- Extraer de ese objeto su partición del batch, pesos, activaciones, preactivaciones, etiquetas, derivadas y metadatos necesarios
- Ejecutar backpropagation local sin realizar forward pass propio
- Calcular gradientes para su partición
- Retornar al master un objeto `GRADIENT_RESULT` con los gradientes calculados

5. Transferencia Confiable de Datos (RDT)

UDP no garantiza la entrega, el orden, la integridad ni la protección contra duplicados. Por esta razón, la implementación agrega una capa RDT sobre UDP basada en Go-Back-N con ACK acumulativo y CRC32.

La capa RDT recibe una secuencia opaca de bytes, la divide en fragmentos DATA, la transmite y reconstruye exactamente la misma secuencia en el receptor. La interpretación de tensores, gradientes, dimensiones y metadatos pertenece a la capa de aplicación y no al protocolo de transferencia.

6. Características del Protocolo RDT

Característica	Descripción
ACK acumulativo	Confirma el último DATA recibido correctamente y en orden
Ventana Go-Back-N	Permite mantener hasta 8 datagramas DATA en tránsito
Número de secuencia	Identifica la posición de cada fragmento DATA dentro de la transferencia
Timeout	Detecta la ausencia de un ACK esperado
Retransmisión Go-Back-N	Retransmite desde el primer DATA todavía no confirmado
CRC32	Detecta corrupción en el encabezado y en el payload transmitido
Recepción en orden	El receptor acepta únicamente el DATA con el número de secuencia esperado
Control de transferencia	START inicia la recepción y END confirma que el objeto completo fue reconstruido

El protocolo no utiliza NACK explícitos. Ante un DATA duplicado o fuera de orden, el receptor vuelve a enviar el último ACK acumulativo válido. Ante un datagrama corrupto proveniente del emisor de una transferencia ya iniciada, también reenvía ese último ACK. Si todavía no aceptó ningún DATA, utiliza el valor especial `ACK_NONE`.

Parámetro	Valor implementado
Endianness del encabezado	Big-endian / network byte order
Tamaño máximo del datagrama UDP	500 bytes
Tamaño fijo del encabezado	28 bytes
Tamaño máximo del payload	472 bytes
Tamaño de ventana Go-Back-N	8 datagramas DATA
Timeout de retransmisión	500 ms
Máximo de reintentos	5
Primer número de secuencia DATA	0
ACK sin DATA confirmado	ACK_NONE = 0xFFFFFFFF
ID del master	0
IDs de workers	1 a 10
Puertos predeterminados de workers	9001 a 9010
Espera para iniciar respuesta del worker	60 segundos después de terminar el envío del master

7. Parámetros del Protocolo

Los campos numéricos de más de un byte se serializan en big-endian. Los datagramas tienen longitud variable: contienen siempre los 28 bytes del encabezado y solamente los bytes válidos del payload. Por ello, START, ACK y END ocupan 28 bytes, mientras que un DATA puede ocupar hasta 500 bytes.

8. Formato del Datagrama UDP

8.1. Estructura del datagrama

Campo	Tamaño	Offset inicial
Tipo de datagrama	1 byte	0
Número de secuencia	4 bytes	1
Número ACK	4 bytes	5
ID de transferencia	4 bytes	9
ID del emisor	2 bytes	13
ID del receptor	2 bytes	15
Tipo de objeto	1 byte	17
Tamaño total del objeto	4 bytes	18
Tamaño útil del payload	2 bytes	22
CRC32	4 bytes	24
Payload	0 a 472 bytes	28

El encabezado no contiene campos separados para número de fragmento ni total de fragmentos. El número de secuencia identifica directamente cada fragmento DATA. La cantidad total de fragmentos se deriva del tamaño total del objeto y del máximo de 472

bytes por payload. Incluso un objeto vacío utiliza una transferencia con un DATA de payload vacío.

8.2. Descripción de campos

- **Tipo de datagrama:** Identifica DATA, ACK, START o END.
- **Número de secuencia:** En DATA indica el índice del fragmento. En END contiene la cantidad total de fragmentos.
- **Número ACK:** En ACK indica el último DATA recibido correctamente y en orden. El ACK final de END contiene la cantidad total de fragmentos.
- **ID de transferencia:** Identifica una transferencia lógica completa.
- **ID del emisor y receptor:** Identifican los extremos esperados de la transferencia.
- **Tipo de objeto:** Identifica la dirección y propósito general del payload para la capa de transporte.
- **Tamaño total del objeto:** Indica cuántos bytes debe contener la reconstrucción final.
- **Tamaño útil del payload:** Indica cuántos bytes siguen al encabezado; no puede superar 472.
- **CRC32:** Permite validar el encabezado y el payload realmente transmitido.
- **Payload:** Contiene un fragmento opaco del objeto; solamente DATA transporta payload.

9. Tipos de Datagrama

Tipo	Valor numérico	Significado
DATA	0x01	Transporta un fragmento del objeto
ACK	0x02	Confirma START, DATA o END mediante el campo ACK
START	0x03	Inicia la recepción cuando aún no hay una transferencia activa
END	0x04	Solicita validar y finalizar la transferencia reconstruida

10. Tipos de Objeto del Protocolo

Los nombres `WORK_ASSIGNMENT` y `GRADIENT_RESULT` corresponden a conceptos de la capa de aplicación. Para la capa RDT, esos contenidos se transportan respectivamente como `WORKER_PAYLOAD` y `MASTER_PAYLOAD`, sin interpretar su estructura interna. El concepto `CONTROL` puede conservarse como una posible extensión de la capa de aplicación, pero no está definido ni es aceptado como tipo de objeto por la implementación actual del protocolo RDT.

Objeto	Valor numérico	Significado
NONE	0x00	Tipo utilizado por los datagramas ACK
WORKER_PAYLOAD	0x01	Secuencia opaca de bytes enviada por el master hacia un worker
MASTER_PAYLOAD	0x02	Secuencia opaca de bytes enviada por un worker hacia el master

11. Capa de Aplicación y Objetos Serializados

El protocolo RDT no interpretará tensores, pesos, activaciones, etiquetas ni gradientes. Su responsabilidad será transportar de forma confiable un objeto serializado completo, identificado por un único ID de transferencia.

La semántica de los datos estará definida en una capa superior de aplicación:

- **WORK_ASSIGNMENT**: objeto creado por el master para un worker específico. Incluye la partición del batch global, pesos, activaciones, preactivaciones, etiquetas, derivadas de pérdida y metadatos necesarios para ejecutar backpropagation local.
- **GRADIENT_RESULT**: objeto creado por un worker para el master. Incluye los gradientes calculados sobre su partición y los metadatos necesarios para validar forma, dtype, capa y parámetro asociado.

Cada objeto será serializado por la biblioteca C++ en una secuencia binaria. Luego el protocolo RDT fragmentará esa secuencia en datagramas DATA, los transmitirá usando Go-Back-N y reconstruirá exactamente el mismo objeto en el receptor. Una vez reconstruido, la capa de aplicación deserializará el objeto y procesará sus tensores internos.

12. Identificadores de Transferencia

Cada dirección de comunicación utiliza un ID de transferencia distinto para evitar que los datagramas de envío y respuesta se mezclen:

- Master hacia worker: $1000 + \text{worker_id}$.
- Worker hacia master: $2000 + \text{worker_id}$.

Un datagrama solamente es procesado por el receptor correspondiente cuando coinciden su ID de transferencia, ID de emisor, ID de receptor y tipo de objeto. El master también verifica que la dirección IP y el puerto de origen correspondan al worker esperado.

13. Campos Usados por Cada Tipo de Datagrama

Tipo de datagrama	Campos principales utilizados
-------------------	-------------------------------

START	Tipo, ID de transferencia, IDs de emisor y receptor, tipo de objeto, tamaño total del objeto y CRC32. Si la transferencia todavía no está activa, el receptor responde con <code>ACK_NONE</code> .
DATA	Tipo, número de secuencia, ID de transferencia, IDs de emisor y receptor, tipo de objeto, tamaño total del objeto, tamaño útil del payload, CRC32 y payload.
ACK	Tipo, número ACK, ID de transferencia, IDs de emisor y receptor y CRC32. Utiliza tipo de objeto <code>NONE</code> , tamaño de objeto cero y payload vacío.
END	Tipo, número de secuencia igual a la cantidad total de fragmentos, ID de transferencia, IDs de emisor y receptor, tipo de objeto, tamaño total del objeto y CRC32.

14. Integridad de Datagramas

Cada datagrama incluye un CRC32 calculado sobre sus 28 bytes de encabezado y su payload válido. Para calcularlo, el campo CRC32 se coloca temporalmente en cero:

$$\text{CRC32} = \text{CRC32}(\text{encabezado con CRC32} = 0 + \text{payload válido})$$

Al recibir un datagrama también se valida que su tipo y tipo de objeto sean reconocidos, que el tamaño útil no supere 472 bytes y que la longitud recibida sea exactamente igual a 28 bytes más el tamaño útil declarado. Si cualquiera de estas comprobaciones falla, el datagrama se considera corrupto y se descarta.

Si el receptor ya inició la transferencia y el datagrama corrupto proviene del mismo extremo, responde con el último ACK acumulativo válido. Antes de recibir START no existe una dirección de transferencia confirmada y no se responde a un datagrama corrupto.

15. Números de Secuencia, ACK y Ventana

Los DATA se numeran desde 0 hasta $N - 1$, donde N es la cantidad de fragmentos derivada del tamaño total del objeto. El receptor mantiene el próximo número esperado y solamente agrega el DATA que coincide exactamente con ese valor.

El ACK es acumulativo:

- Un ACK con valor n confirma todos los DATA desde 0 hasta n .
- `ACK_NONE` confirma START e indica que todavía no existe un DATA aceptado.
- Un DATA duplicado o fuera de orden se descarta y provoca el reenvío del último ACK válido.
- El ACK de END utiliza el valor N , uno después del último número de secuencia DATA.

El emisor mantiene una ventana fija de 8 DATA. Al recibir un ACK válido mueve la base de la ventana al datagrama siguiente. Si la ventana queda totalmente confirmada, continúa con la siguiente ventana.

16. Timeouts y Reintentos

Cuando el emisor espera confirmación de START, de una ventana DATA o de END, utiliza un timeout de 500 ms. Si el timeout expira, incrementa el contador de reintentos:

- En START, vuelve a enviar START.
- En DATA, vuelve a enviar desde el primer DATA no confirmado.
- En END, vuelve a enviar END.

Un ACK válido reinicia el contador de reintentos. Si el contador supera el máximo de 5 reintentos, el emisor marca la transferencia como fallida.

17. Simulación de Fallos

El protocolo normal no altera los datagramas. Para comprobar su capacidad de recuperación se puede activar una simulación opcional antes de ejecutar el master y los workers:

```
RDT_SIMULATION=1 RDT_SIMULATION_SEED=42 ./master ...  
RDT_SIMULATION=1 RDT_SIMULATION_SEED=42 ./worker ...
```

RDT_SIMULATION=1 activa la simulación. RDT_SIMULATION_SEED es opcional y permite repetir la misma selección aleatoria de fallos. Para obtener una ejecución reproducible se debe usar la misma semilla en el master y en todos los workers.

Cada transferencia selecciona aleatoriamente entre uno y dos datagramas para cada uno de los siguientes casos:

- **Delay:** el datagrama se envía de forma diferida después de una demora aleatoria entre 100 y 900 ms.
- **Pérdida:** el datagrama no se envía y el emisor debe recuperarlo mediante timeout y retransmisión.
- **Corrupción:** se modifica un bit después de calcular CRC32; el receptor detecta que el CRC no coincide y descarta el datagrama.

La selección puede afectar START, DATA, END, ACK_NONE, los ACK de DATA o el ACK final. Los eventos se aplican una sola vez al datagrama seleccionado, por lo que una retransmisión posterior puede completar la transferencia. Durante la simulación el máximo de reintentos aumenta de 5 a 20; con la simulación desactivada se conserva el límite normal de 5.

La consola indica cada evento y sus consecuencias. Por ejemplo:

```
SIMULATION DROP type=DATA seq=3 transfer_id=1001  
SIMULATION DELAY type=ACK ack=2 transfer_id=1001 delay_ms=734  
SIMULATION CORRUPT type=DATA seq=5 transfer_id=1001 byte_offset=31 bit=2  
RECEIVED CORRUPT_DATAGRAM reason=CRC_MISMATCH  
TIMEOUT transfer_id=1001 phase=DATA retry=2 max_retries=20  
SENT_DELAYED type=ACK ...
```

18. Estados del Emisor y Receptor

18.1. Estados del Emisor

Estado	Descripción
START	Envía START y espera un ACK con valor ACK_NONE
DATA	Envía y confirma las ventanas Go-Back-N
END	Envía END con secuencia N y espera un ACK con valor N
DONE	La transferencia terminó correctamente
FAILED	El envío falló o superó el máximo de reintentos

18.2. Estados Lógicos del Receptor

Estado	Descripción
ESPERANDO	Espera un START que coincida con los identificadores de la transferencia
RECIBIENDO	Acepta únicamente el DATA esperado y mantiene el último ACK acumulativo
FINALIZADO	Validó END, la cantidad de fragmentos y el tamaño exacto del objeto reconstruido

19. Sesiones Master–Worker

El master requiere exactamente diez workers con IDs únicos del 1 al 10. La implementación predeterminada utiliza 127.0.0.1 y los puertos 9001 a 9010. Cada worker abre un socket UDP en el puerto 9000 + `worker_id`.

El master crea un thread y un socket UDP independiente por worker. Cada sesión mantiene su propio emisor para `WORKER_PAYLOAD` y su propio receptor para `MASTER_PAYLOAD`. Después de enviar completamente el payload del worker, el master espera hasta 60 segundos para que el worker inicie su respuesta. Las sesiones continúan de forma independiente; al finalizar todos los threads, el master reúne los payloads recibidos y reporta juntos los errores de las sesiones que fallaron.

El worker recibe primero un `WORKER_PAYLOAD`, guarda la dirección del socket del master que inició la transferencia y utiliza esa misma dirección y su mismo socket para enviar el `MASTER_PAYLOAD`.

20. Secuencia de Comunicación

Cada objeto se transfiere mediante la siguiente secuencia:

1. El emisor envía START con el tamaño total del objeto.

2. El receptor guarda la dirección del emisor, inicializa su estado de recepción, deriva N desde el tamaño total y responde con `ACK_NONE`.
3. El emisor transmite DATA desde la secuencia 0 usando ventanas de hasta 8 datagramas.
4. Por cada DATA esperado, el receptor agrega su payload y responde con un ACK acumulativo.
5. Ante DATA duplicado, fuera de orden o corrupto, el receptor descarta el datagrama y reenvía el último ACK válido.
6. Ante timeout, el emisor retransmite START, END o todos los DATA desde el primero todavía no confirmado.
7. Después de confirmar los N DATA, el emisor envía END con número de secuencia N .
8. El receptor verifica que recibió N fragmentos y exactamente el tamaño de objeto anunciado. Si la validación es correcta, responde con ACK igual a N y finaliza.
9. El emisor recibe el ACK final y marca la transferencia como completada.

21. Distribución del Batch y Objetos

El servidor principal coordinará la distribución del aprendizaje. Para ello invocará un programa o módulo de distribución que tomará un único batch global y lo dividirá en 11 partes iguales.

- La partición 0 será usada por el master.
- Las particiones 1 a 10 serán enviadas a los workers 1 a 10.
- Los workers podrán ejecutarse en computadoras distintas o en terminales separadas.

El forward pass se realizará únicamente en el master sobre el batch global. Durante ese forward pass, el master guardará los estados intermedios necesarios para backpropagation.

La red neuronal tendrá 4 capas ocultas de 200 neuronas. Por lo tanto, la implementación base utilizará 4 matrices principales de pesos:

$$W_1, W_2, W_3, W_4$$

Cada matriz tendrá tamaño 200x200. Sin embargo, el protocolo no se limita a enviar estas matrices. El master encapsulará en un objeto `WORK_ASSIGNMENT` todo dato requerido para ejecutar backpropagation de forma correcta sobre la partición del worker, incluyendo:

- La partición correspondiente del batch global.
- Las matrices de pesos requeridas por la red.
- Activaciones generadas por el forward pass del master.
- Preactivaciones necesarias para derivadas de funciones de activación.

- Etiquetas o targets de la partición.
- Derivadas iniciales de la función de pérdida, cuando corresponda.
- Metadatos de forma, dtype, capa, partición y reconstrucción.

Si la red utiliza biases u otros parámetros entrenables, estos se incluirán dentro del mismo objeto de aplicación y se tratarán con el mismo mecanismo de tensores y gradientes.

22. Serialización de Objetos

Los objetos de aplicación serán serializados como una secuencia binaria única. Dentro de cada objeto, las particiones del batch y los tensores se almacenarán como arreglos continuos en formato little-endian. El tipo de dato principal será float32, pero cada objeto incluirá metadatos que indiquen dtype, dimensiones, tipo de tensor, capa, parámetro asociado y partición.

Python enviará arreglos NumPy hacia C++ mediante pybind11.

C++ convertirá cada objeto en una secuencia binaria para fragmentarla en datagramas UDP de hasta 500 bytes.

Cada worker reconstruirá el objeto completo antes de deserializarlo e iniciar su cálculo local de backpropagation.

23. Flujo General del Sistema

1. Python carga el dataset y define la red neuronal.
2. Python selecciona un único batch global de entrenamiento.
3. El servidor principal invoca el programa de distribución del batch.
4. El batch global se divide en 11 partes iguales.
5. El master conserva la partición 0 y asigna las otras 10 particiones a los workers.
6. El master ejecuta el forward pass sobre el batch global completo.
7. El master guarda pesos, activaciones, preactivaciones, etiquetas, derivadas iniciales y metadatos necesarios.
8. Python construye un objeto `WORK_ASSIGNMENT` para cada worker y lo entrega a la biblioteca C++ mediante pybind11.
9. C++ serializa y transmite cada objeto `WORK_ASSIGNMENT` usando UDP con Go-Back-N.
10. Cada worker reconstruye y deserializa su objeto `WORK_ASSIGNMENT`.
11. Cada worker ejecuta backpropagation local sobre su partición sin realizar forward pass propio.
12. El master ejecuta backpropagation local sobre la partición 0.

13. Cada worker construye un objeto `GRADIENT_RESULT` con sus gradientes y llama a la biblioteca C++ para enviarlo al master.
14. El master reconstruye y deserializa los objetos `GRADIENT_RESULT` de los 10 workers.
15. El master promedia sus gradientes locales con los gradientes recibidos.
16. Python actualiza la red neuronal principal aplicando los gradientes promediados.

24. Promedio de Gradientes y Actualización de Pesos

La operación principal de combinación será el promedio de gradientes por capa. Para cada capa oculta l , el master calculará:

$$G_l^{avg} = \frac{G_l^{master} + \sum_{i=1}^{10} G_l^{worker_i}}{11}$$

Este promedio se realizará para los gradientes asociados a las 4 matrices principales de pesos:

$$l \in \{1, 2, 3, 4\}$$

Luego, Python actualizará los pesos de la red principal usando una regla de optimización. En el caso de SGD simple:

$$W_l^{new} = W_l^{old} - \eta G_l^{avg}$$

Si la implementación incluye biases u otros parámetros entrenables, el mismo esquema se aplicará a sus gradientes correspondientes.

25. Integración Python y C++

El proyecto utilizará pybind11 para conectar Python con C++.
Ejemplo:

```
import distributed_nn

pesos_actualizados = distributed_nn.run_training_round(
    dataset,
    pesos_iniciales,
    workers
)
```

Python enviará arreglos NumPy hacia C++.

C++ no definirá la red neuronal. La biblioteca C++ se encargará de serializar objetos de aplicación, fragmentarlos, enviar y recibir datagramas UDP, aplicar Go-Back-N, verificar CRC32, administrar ACK acumulativos, reconstruir objetos completos y devolver hacia Python los gradientes contenidos en los objetos `GRADIENT_RESULT`.

26. Posibles Mejoras Futuras

- Selective Repeat
- Timeout dinámico
- Paralelismo configurable para más workers
- Compresión de datagramas
- Balanceo dinámico de carga
- Aceleración GPU
- Paralelismo híbrido CPU/GPU

27. Tecnologías Utilizadas

Las tecnologías utilizadas son las siguientes:

Tecnología	Uso
Python	Definición y entrenamiento de la red neuronal
PyTorch	Machine Learning
C++	Biblioteca de comunicación UDP/RDT y serialización
UDP	Comunicación
pybind11	Integración Python/C++

28. Conclusión

El proyecto implementa un sistema de entrenamiento distribuido de una red neuronal utilizando UDP y una capa RDT desarrollada manualmente.

La arquitectura combina Python para la definición y entrenamiento de la red neuronal con una biblioteca C++ encargada de la comunicación confiable sobre UDP.

La confiabilidad será implementada mediante:

- Go-Back-N puro
- Ventanas deslizantes
- ACK acumulativos
- CRC32 aplicado sobre encabezado y payload
- Timeouts
- Retransmisiones automáticas
- Máximo de reintentos
- Estados definidos para emisor y receptor

El sistema permitirá dividir un único batch global entre 1 master y 10 workers, ejecutar el forward pass únicamente en el master, distribuir objetos `WORK_ASSIGNMENT` con los datos necesarios para backpropagation y combinar los gradientes recibidos en objetos `GRADIENT_RESULT` mediante promedio elemento a elemento.

La arquitectura propuesta servirá como base para futuras extensiones orientadas a entrenamiento paralelo de redes neuronales.